

Efficiently Computing Latency Distributions by Combined Performance Evaluation Techniques

Freek van den Berg, Boudewijn R. Haverkort and Jozef Hooman

Valuetools 2015, Berlin, Germany
December 14th-16th, 2015



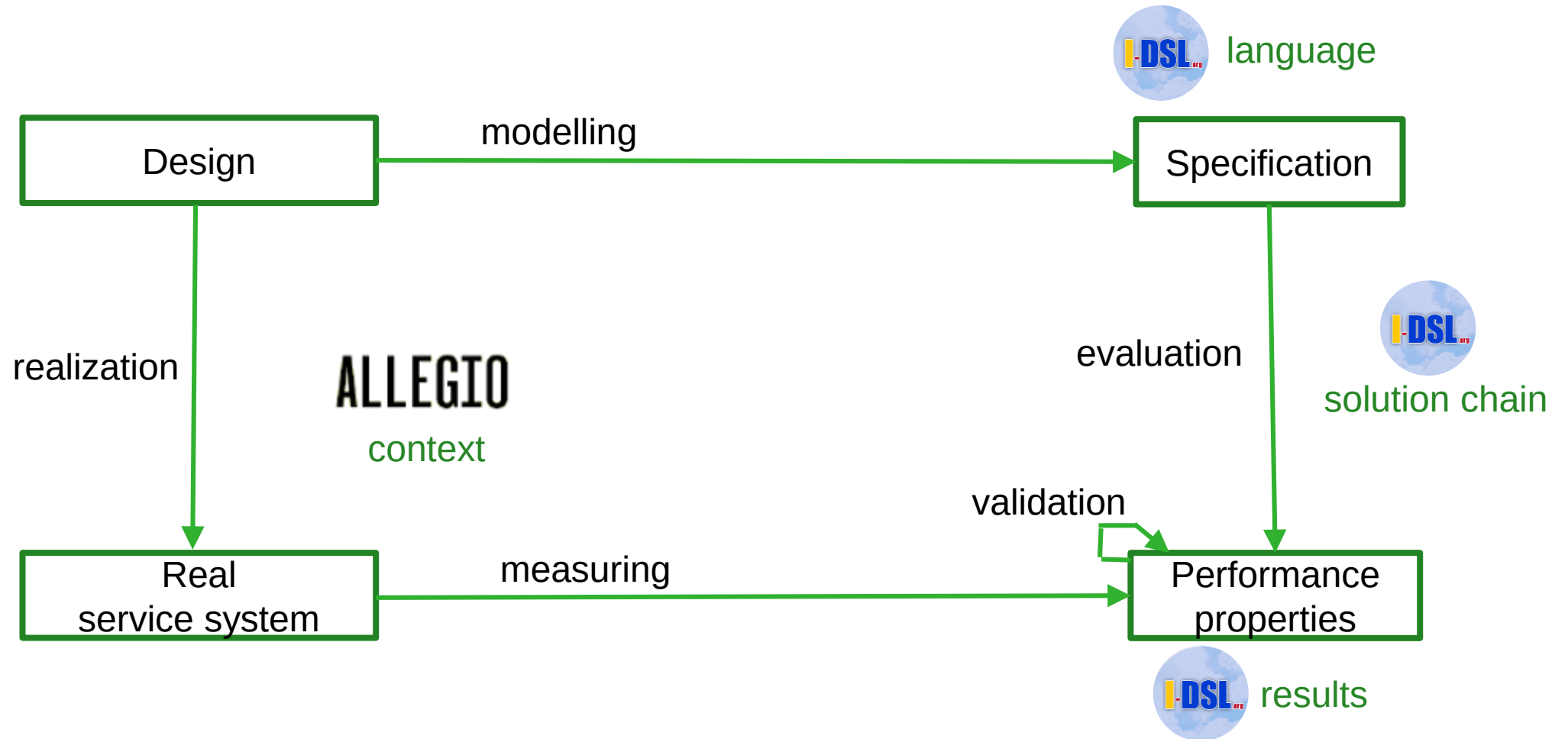
Outline

- Motivation: Computing latency distributions of service systems
- Previous work
 - The iDSL language and toolbox
 - Latency distributions via iterative probabilistic model checking
- Two extensions to iDSL
 - Automatic model simplifications
 - Combined performance evaluation techniques
- Results: Validity and efficiency of the approach

Outline

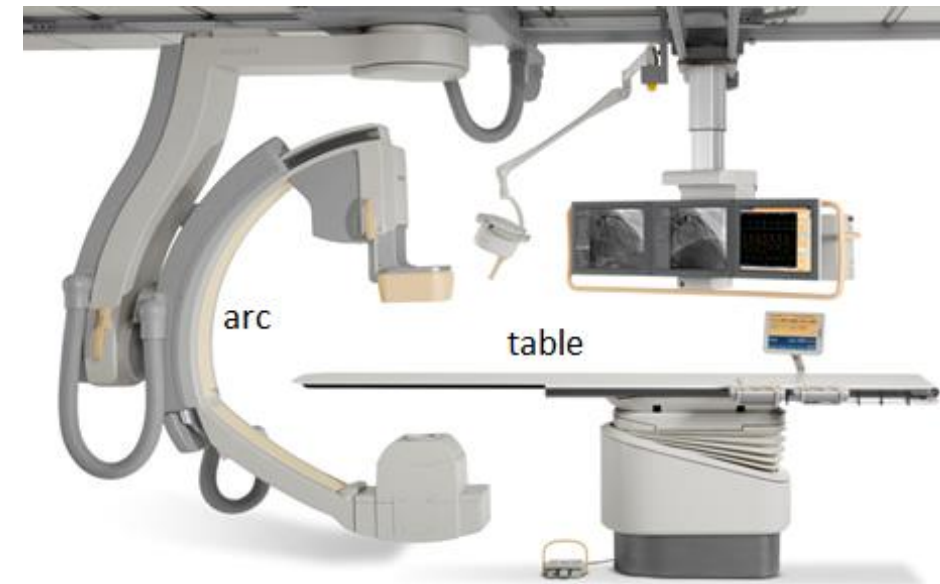
- **Motivation: Computing latency distributions of service systems**
- Previous work
 - The iDSL language and toolbox
 - Latency distributions via iterative probabilistic model checking
- Two extensions to iDSL
 - Automatic model simplifications
 - Combined performance evaluation techniques
- Results: Validity and efficiency of the approach

Performance evaluation of service systems



Allegio: A case study on iXR systems (1/2)

- Medical Imaging Machines (MIS) perform safety critical tasks
- iXR machines are MIS that
 - Dynamically record X-ray based images
 - Used by surgeons while operating a patient
 - Are designed on the basis of measurements
 - Can be depicted as service systems
- Surgeons demand up-to-date images
 - Performance is key for iXR systems
 - Latency is the prime performance metric



Allegio: A case study on iXR systems (2/2)

- Insights in performance needed
 - Key questions:
 - Q1 What is the latency for which a given percentage of service requests completes?
 - Q2 Which percentage of the service requests has a latency below a given value?
 - Simulations show *average behavior*, while WCET methods are *pessimistic*
- Approach
 - Combine performance evaluation techniques to generate **latency distributions**
 - Two automated model simplification strategies

Outline

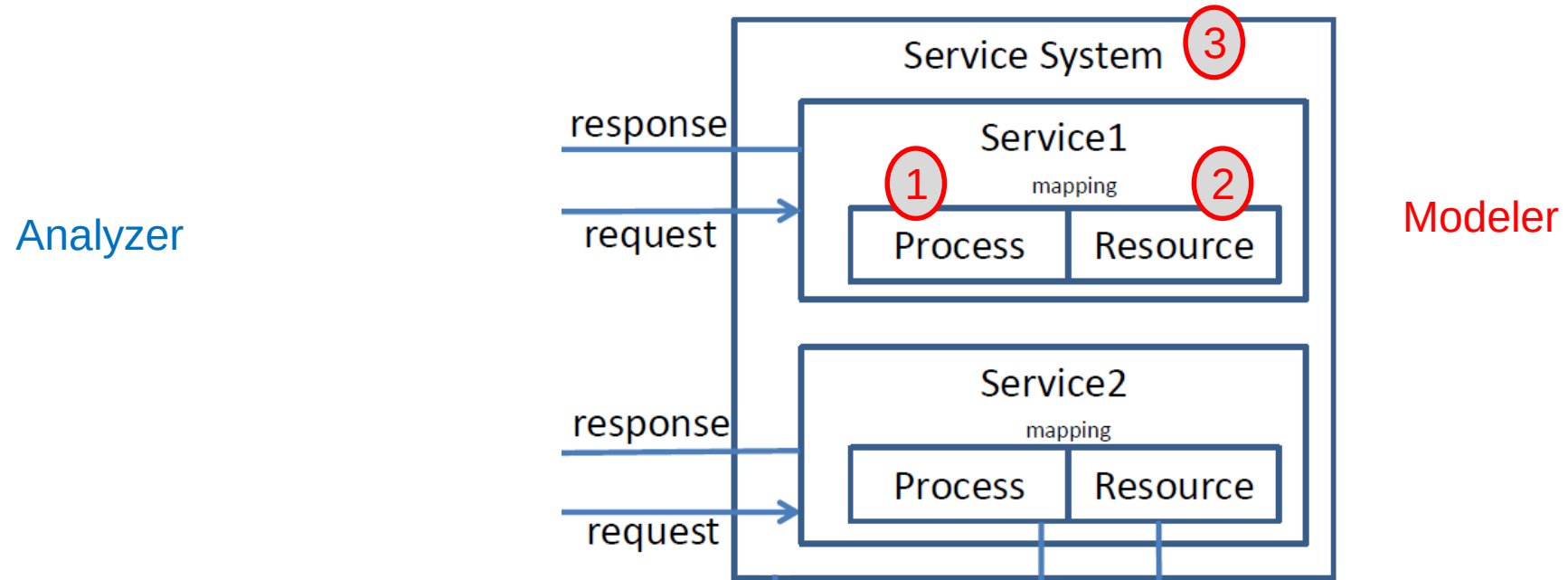
- Motivation: Computing latency distributions of service systems
- **Previous work**
 - **The iDSL language and toolbox**
 - Latency distributions via iterative probabilistic model checking
- Two extensions to iDSL
 - Automatic model simplifications
 - Combined performance evaluation techniques
- Results: Validity and efficiency of the approach

The motivation for iDSL

- Toolsets for modelling and analyzing stochastic systems
 - PRISM + Modest: multiple formalisms and ways of analysis, advanced algorithms
- Drawbacks
 - Hard to use for system designers
 - Low level code: large models, redundancy
 - One model does **not** fit all
 - Limited support for multiple designs, and visualizations
- Our approach
 - A high level language and solution chain for service systems named 
 - Delegate low-level analysis to Modest



The iDSL language model



iDSL language instance of the iXR case study (1/2)

Section Process

```
ProcessModel Image_Processing seq {  
  seq Noise_reduction {  
    atom Pre_processing EDF  
    atom Decompose EDF ...  
    atom Spatial_noise_red EDF ...  
    atom Temporal_noise_red EDF ...  
    atom Compose EDF ...  
  }  
  seq Refinement {  
    alt { atom Refine load EDF ...  
      seq { atom Refine EDF ...  
        atom Refine EDF ...  
      }  
    }  
  }  
}
```

1

Section Resource

```
ResourceModel Image_PC decomp { atom CPU rate 1 }
```

2

Section System

```
Service Frontal_Image_Processing_Service  
  Process Image_Processing  
  Resource Image_PC  
  Mapping assign { ( *,CPU) }  
  scheduling policy { (CPU,FIFO) }  
  
Service Lateral_Image_Processing_Service  
  Process Image_Processing  
  Resource Image_PC  
  Mapping assign { ( *,CPU) }  
  scheduling policy { (CPU,FIFO) }
```

3



iDSL language instance of the iXR case study (2/2)

Section Measure

Measure CDF of ServiceResponse times

4

Section Scenario

Scenario BiPlane_Image_Processing_run

ServiceRequest Frontal_Image_Processing_Service
at time 0, 40000, ...

ServiceRequest Lateral_Image_Processing_Service
at time (0+dspace("offset")),
(40000+dspace("offset")), ...

5

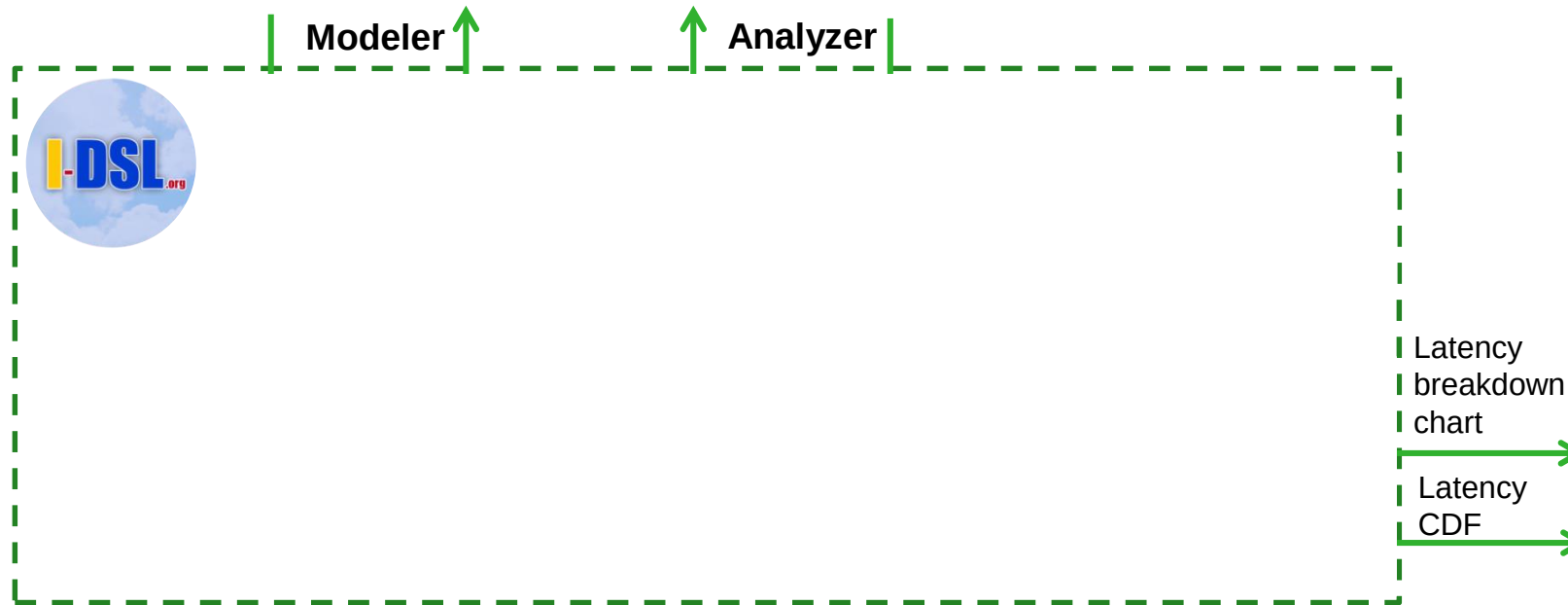
Section Study

Scenario BiPlane_Image_Processing_run

DesignSpace ("offset" {"0" "10000" "20000" "30000"})

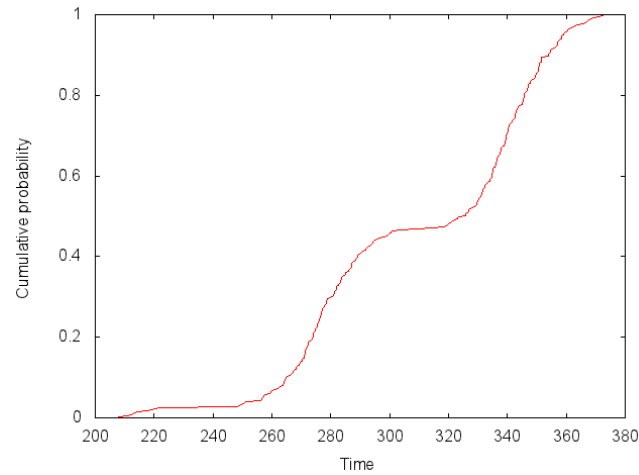
6

iDSL solution chain



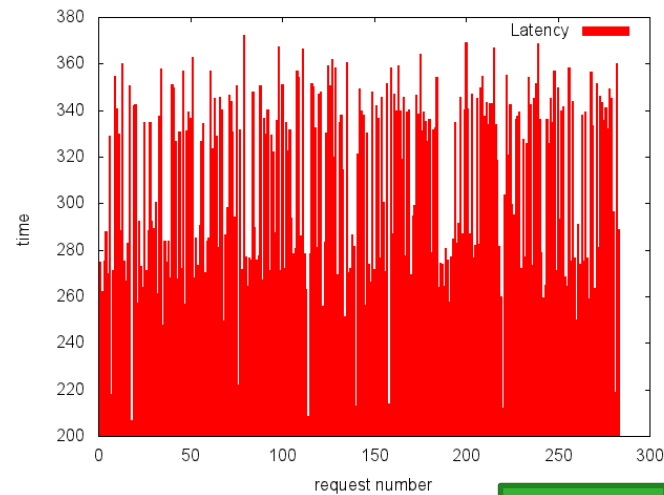
iDSL solution chain output

CDF



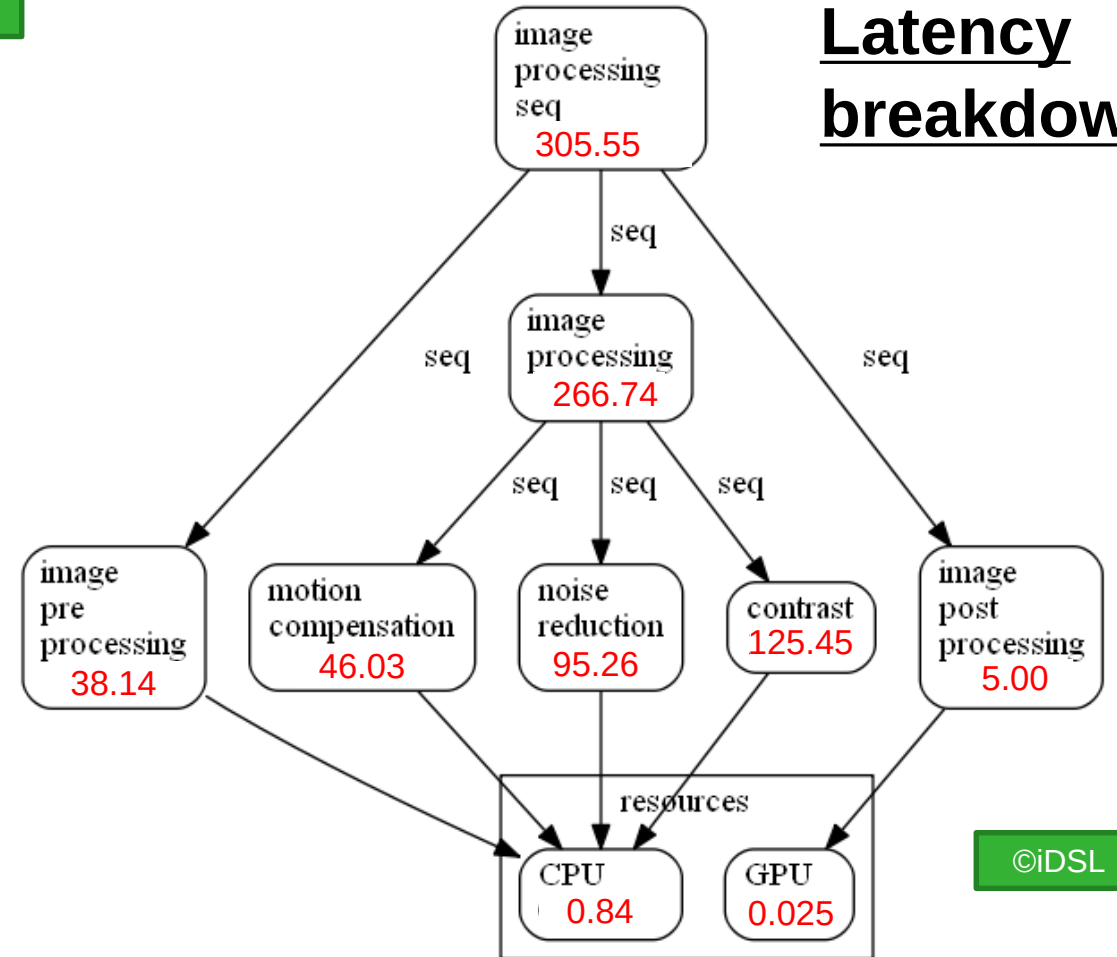
©iDSL

Latency chart



©iDSL

Latency breakdown

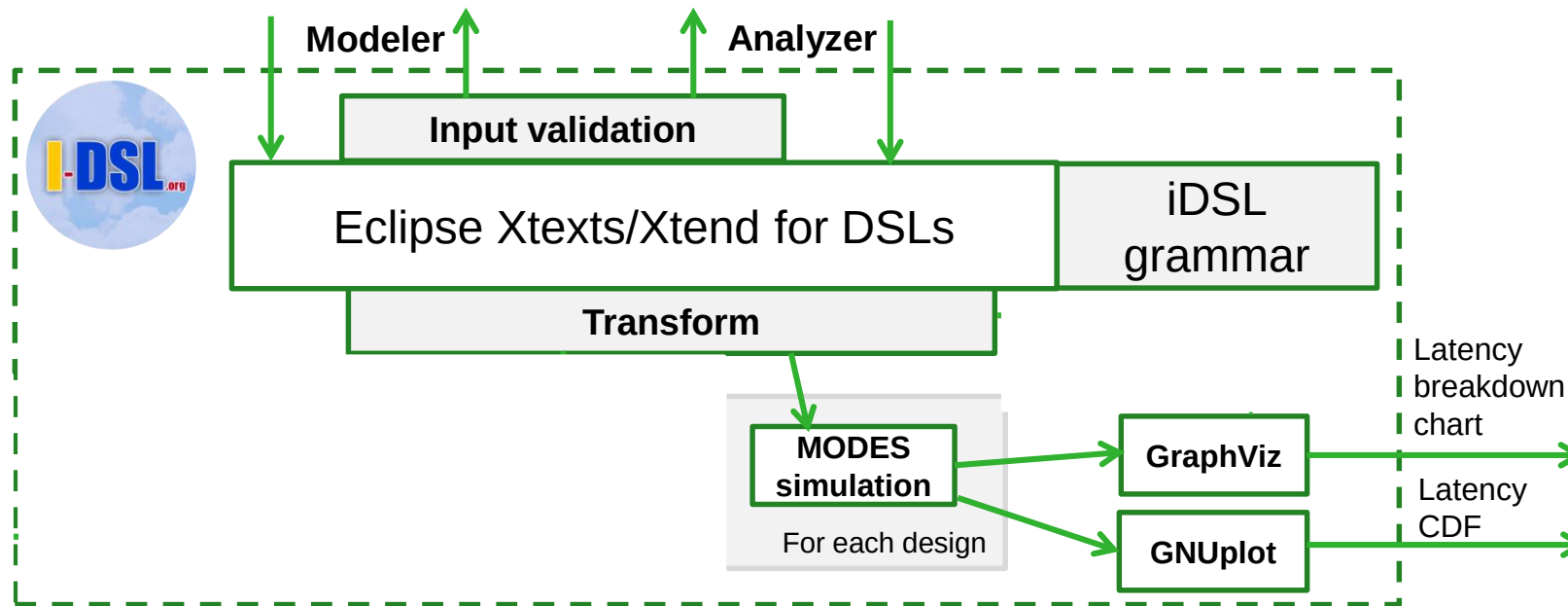


©iDSL

Outline

- Motivation: Computing latency distributions of service systems
- **Previous work**
 - The iDSL language and toolbox
 - **Latency distributions via iterative probabilistic model checking**
- Two extensions to iDSL
 - Automatic model simplifications
 - Combined performance evaluation techniques
- Results: Validity and efficiency of the approach

iDSL solution chain



Probabilistic model checking

- Functions $\mathcal{M}_{min}$ and $\mathcal{M}_{max}$ return the minimum and maximum probability (for all resolutions of nondeterminism) that a service finishes within a given time:

$$p_{min} = \mathcal{M}_{min}(i, s, t)$$

$$p_{max} = \mathcal{M}_{max}(i, s, t)$$

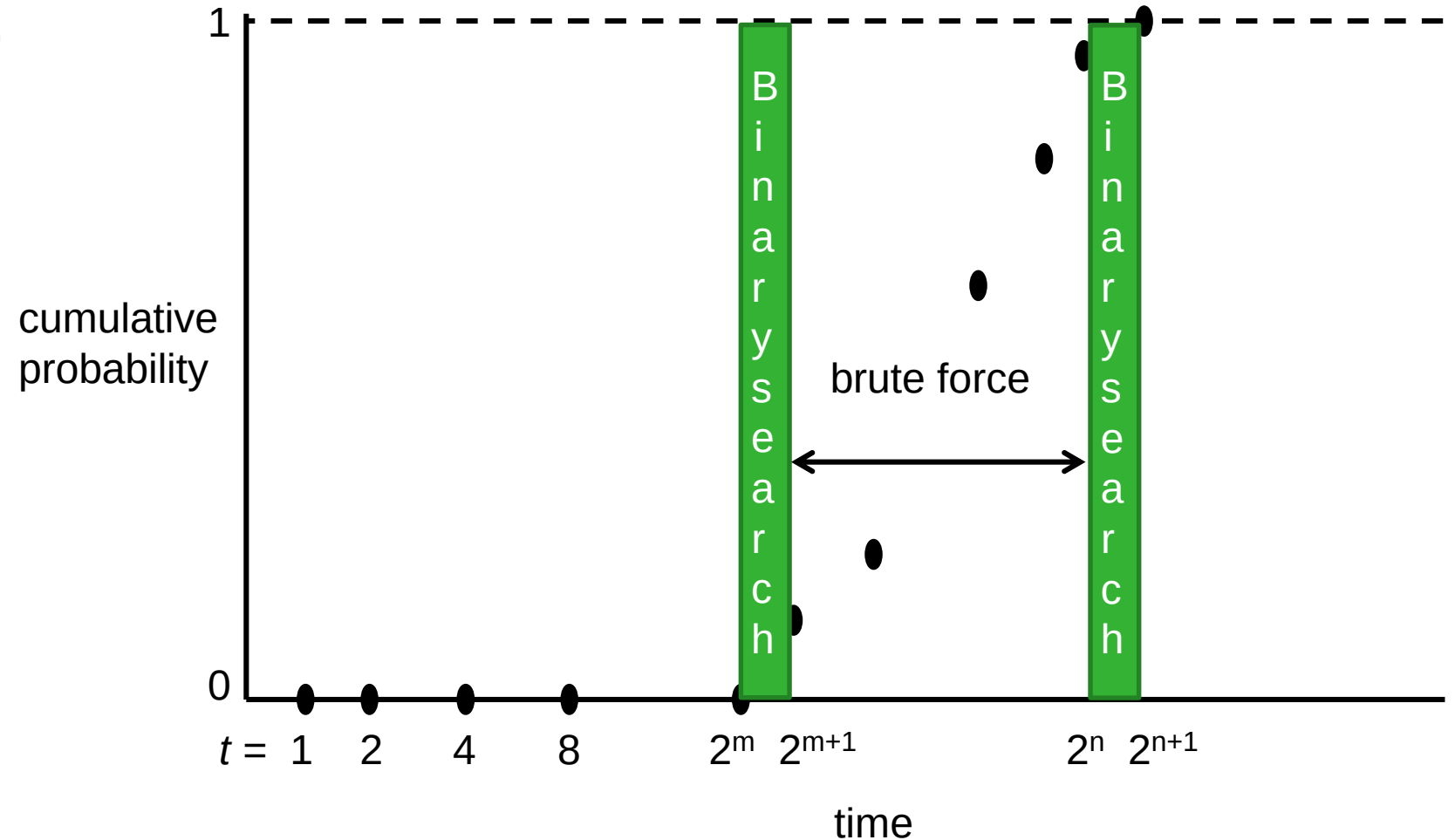
where $\underline{i}$ is an iDSL model,
 $\underline{s}$ a service,
and $\underline{t}$ the given time

Probabilistic model checking, iteratively

$$p_{min} = \mathcal{M}_{min}(i, s, t)$$

We apply
function $\mathcal{M}_{min}$
for different t values

And repeat this for
each model i , service s ,
And function $\mathcal{M}_{max}$



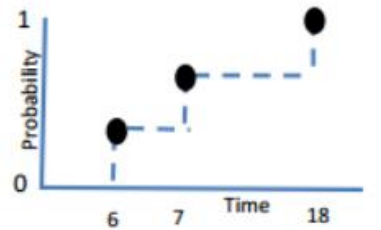
Outline

- Motivation: Computing latency distributions of service systems
- Previous work
 - The iDSL language and toolbox
 - Iterative Probabilistic Model checking
- **Two extensions to iDSL**
 - **Automatic model simplifications**
 - Combined performance evaluation techniques
- Results: Validity and efficiency of the approach

Two orthogonal techniques to simplify system models

- The number of segments per empirical distribution function (EDF) for K-means clustering

For example, a EDF based on measurements 6, 7 and 18



3 clusters

Scanning the models of different complexities

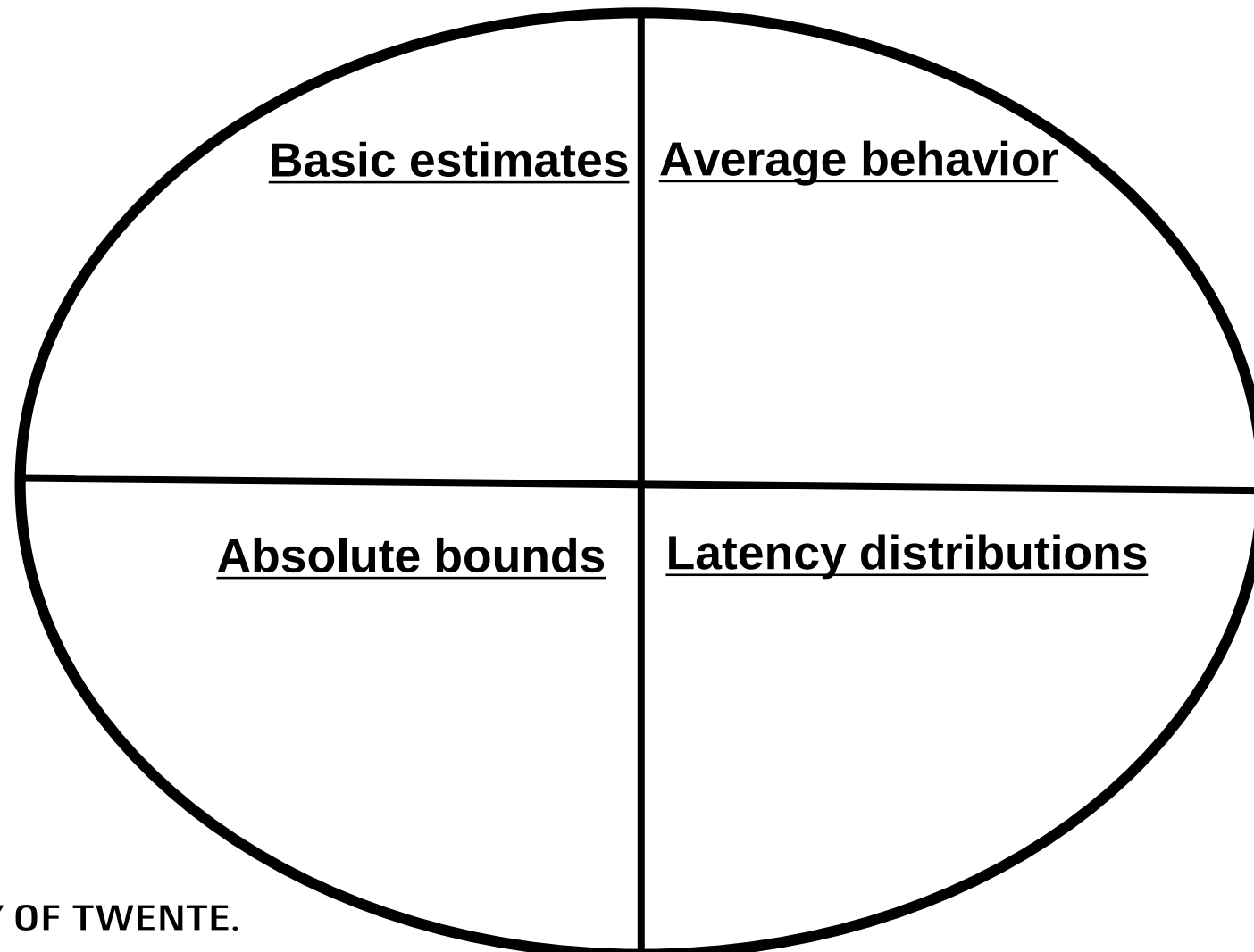
		Model time unit							
Number of clusters		t ₁₆	t ₃₂	t ₆₄	t ₁₂₈	t ₂₅₆	t ₅₁₂	t ₁₀₂₄	loi
	n ₁	>99	40	7	5	3	3	3	.59
	n ₂								.44
	n ₄								.33
	n ₈								.22
	n ₁₆								.15
	n ₃₂								.07
	n ₆₄								0
	lop	8.3	17.5	37.1	80.0	180	377	395	

Execution times of one probabilistic model checking call in iDSL

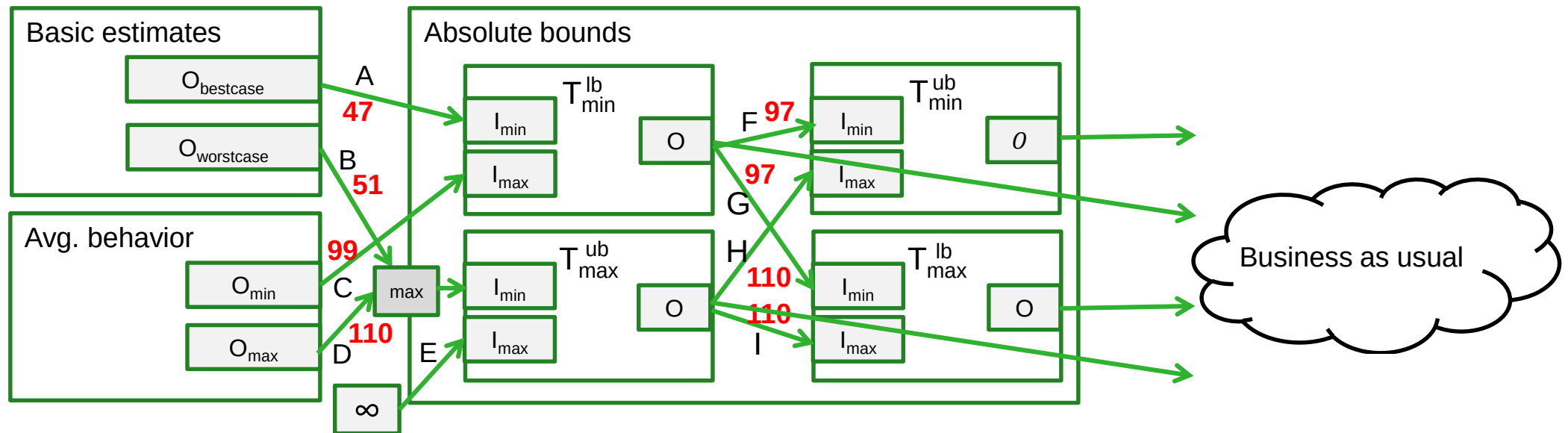
Outline

- Motivation: Computing latency distributions of service systems
- Previous work
 - The iDSL language and toolbox
 - Iterative Probabilistic Model checking
- **Two extensions to iDSL**
 - Automatic model simplifications
 - **Combined performance evaluation techniques**
- Results: Validity and efficiency of the approach

Four performance evaluation techniques



Combining the performance evaluation techniques



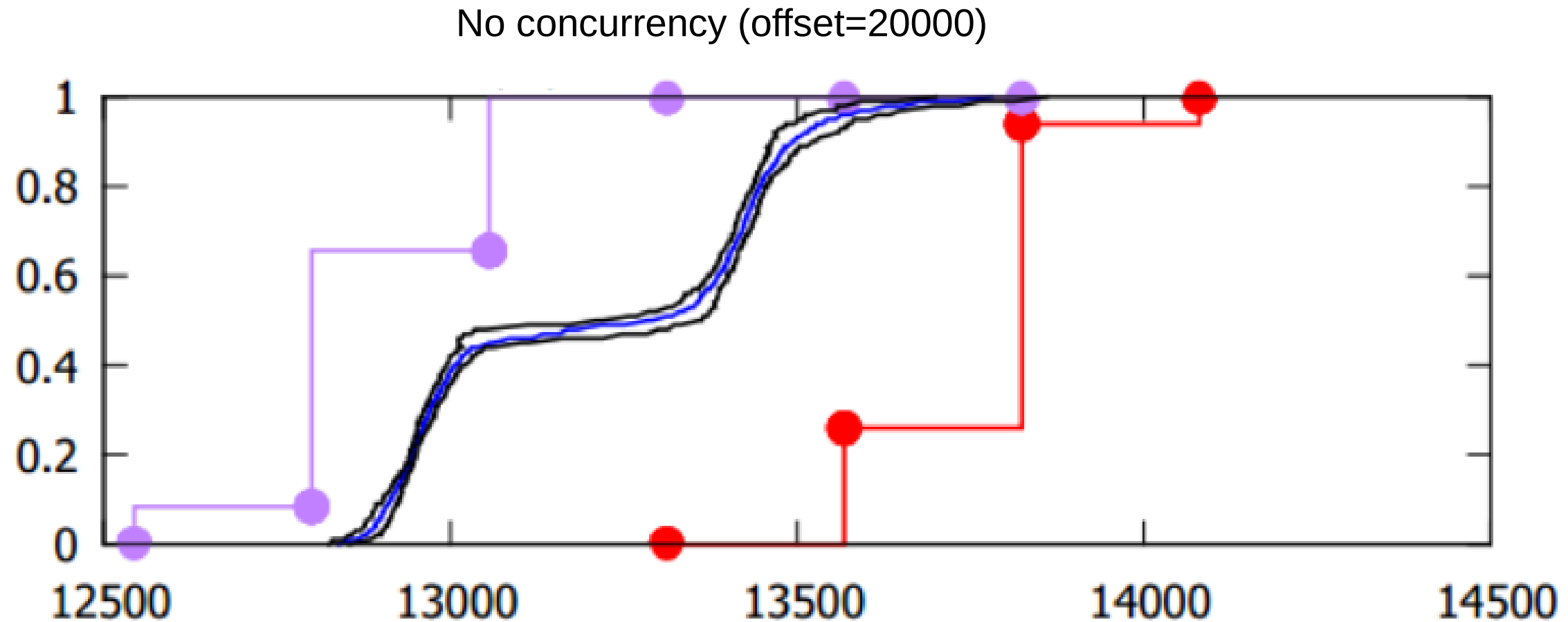
$T_{lb/ub}$ = the minimum or maximum (min/max)
 min/max lower or upper bound (lb/ub) latency for all resolutions of non-determinism

Note: model time unit = 256

Outline

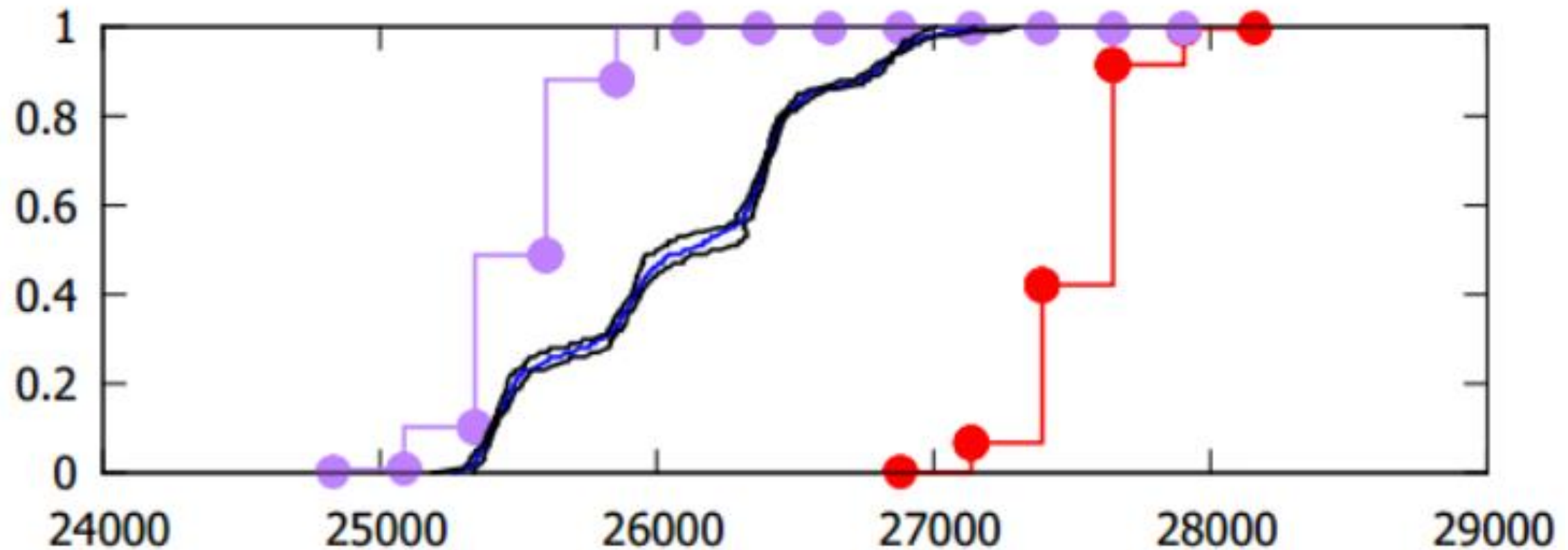
- Motivation: Computing latency distributions of service systems
- Previous work
 - The iDSL language and toolbox
 - Iterative Probabilistic Model checking
- Two extensions to iDSL
 - Automatic model simplifications
 - Combined performance evaluation techniques
- **Results: Validity and efficiency of the approach**

Validity: do the results of the approach match simulation results (0=2000)?



Validity: do the results of the approach match simulation results (0=2000)?

Maximum concurrency (offset=0)



Efficiency: are the extensions made to iDSL useful?

Method	Basic estimates + simulation	Model simplifications	Model checking	Probabilistic model Checking	Σ
No basic estimates and simulations	n/a	499 (16)	1319 (42)	147 (4)	1965 (62)
New method	27	499 (16)	758 (27)	305 (7)	1589 (50)

$758/27=28$

$305/7=44$

Time in seconds
(model checking calls)

Conclusions

- We'd like to compute latency distributions of service systems
 - This is crucial for medical iXR systems
 - Previous work (iDSL) already yielded latency distributions. We want more efficiency and automation
- Therefore, we extended previous work in two ways
 - Combining performance evaluation techniques
 - Basic estimations + simulation: reduces model checking time
 - Model checking is faster than probabilistic model checking
 - Two automated model simplification strategies
 - Takes up much of the total execution time of iDSL
 - Saves manual, error-prone work and leads to a valid model

Thank you for your attention